

Sistem Programlama: C String İşlemlerinin Anatomisi

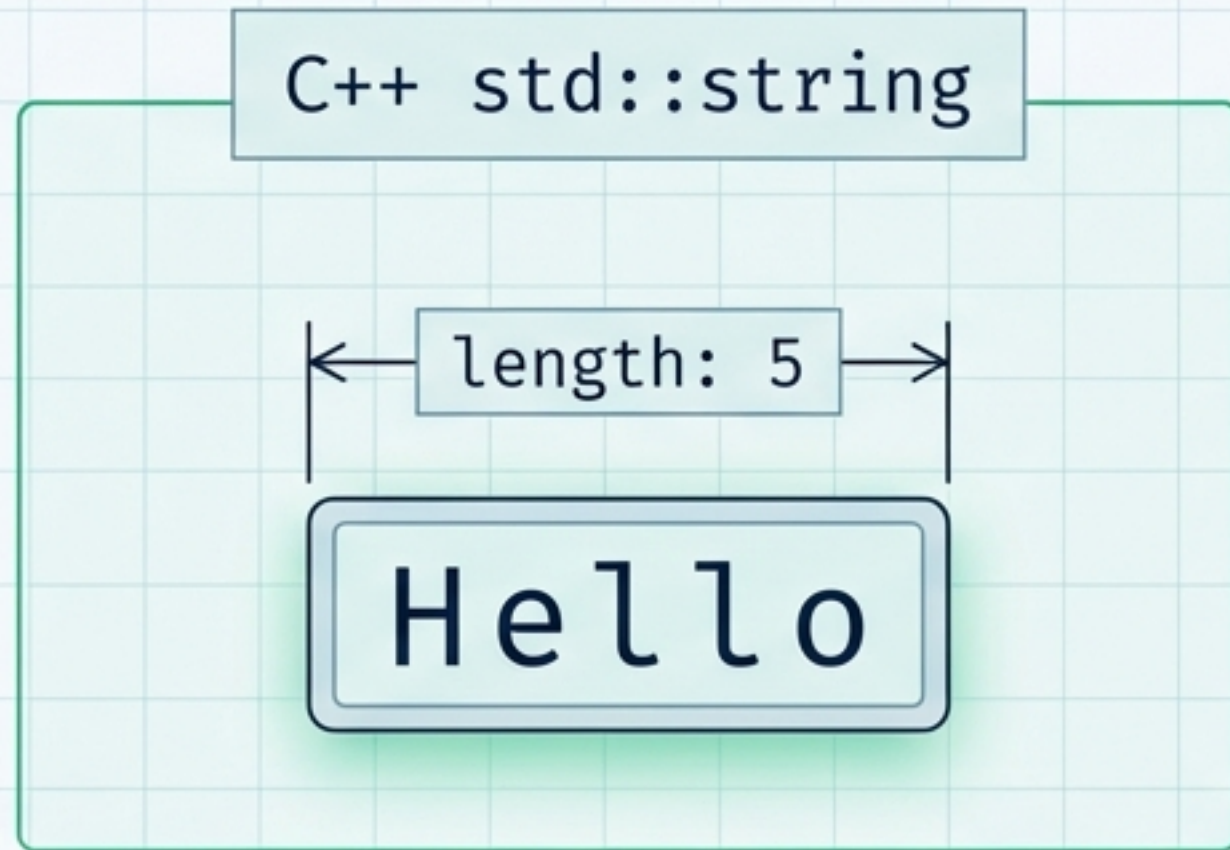
Keskin Araçlar, Gizli Tuzaklar ve Bellek Mimarisi

>_

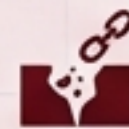
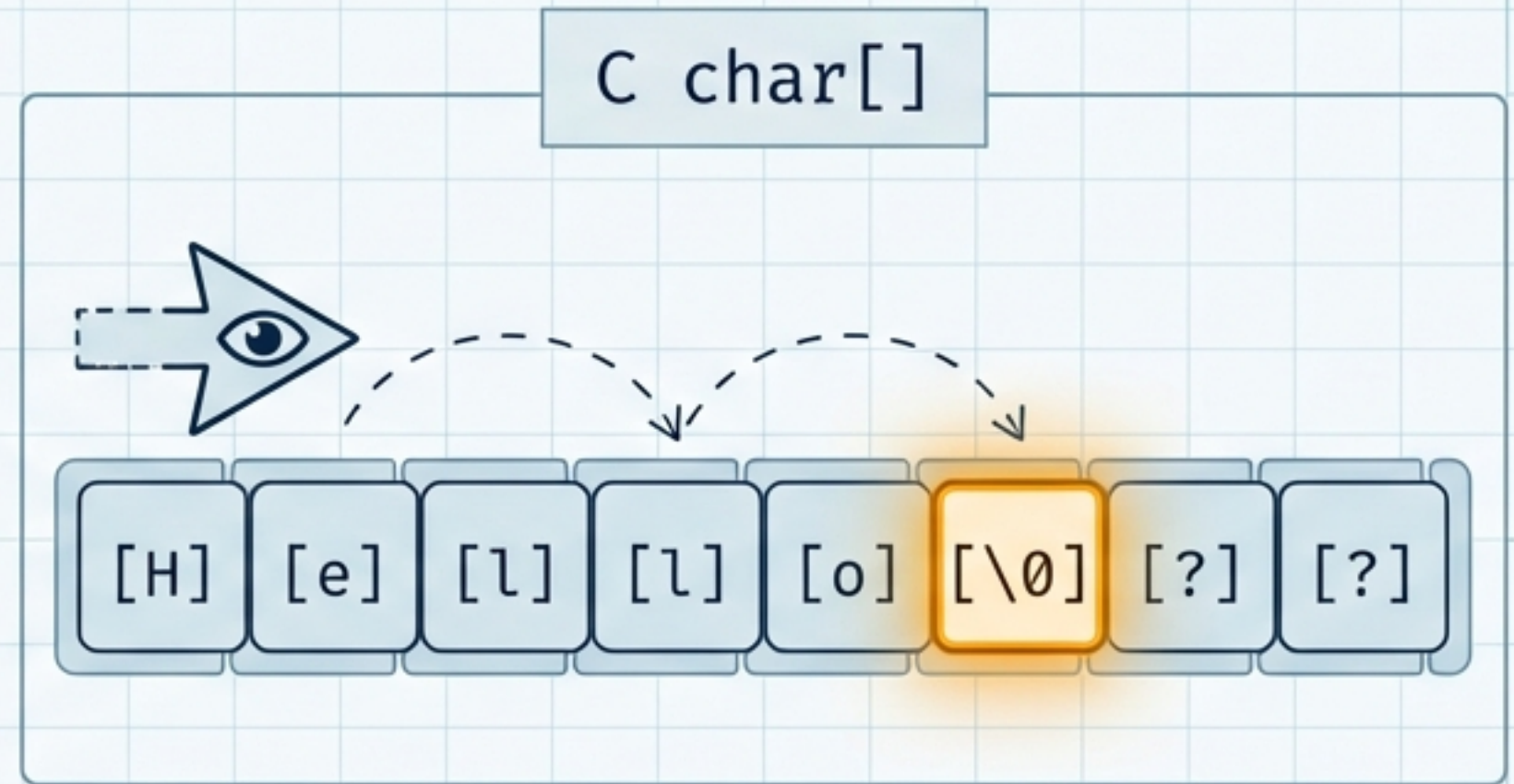
— ×

```
root@sys:~/memory_layout# ./execute_strings
```

Karanlıkta İlerleyen Karakterler



C++ stringleri kendi uzunluklarını ve sınırlarını bilir.

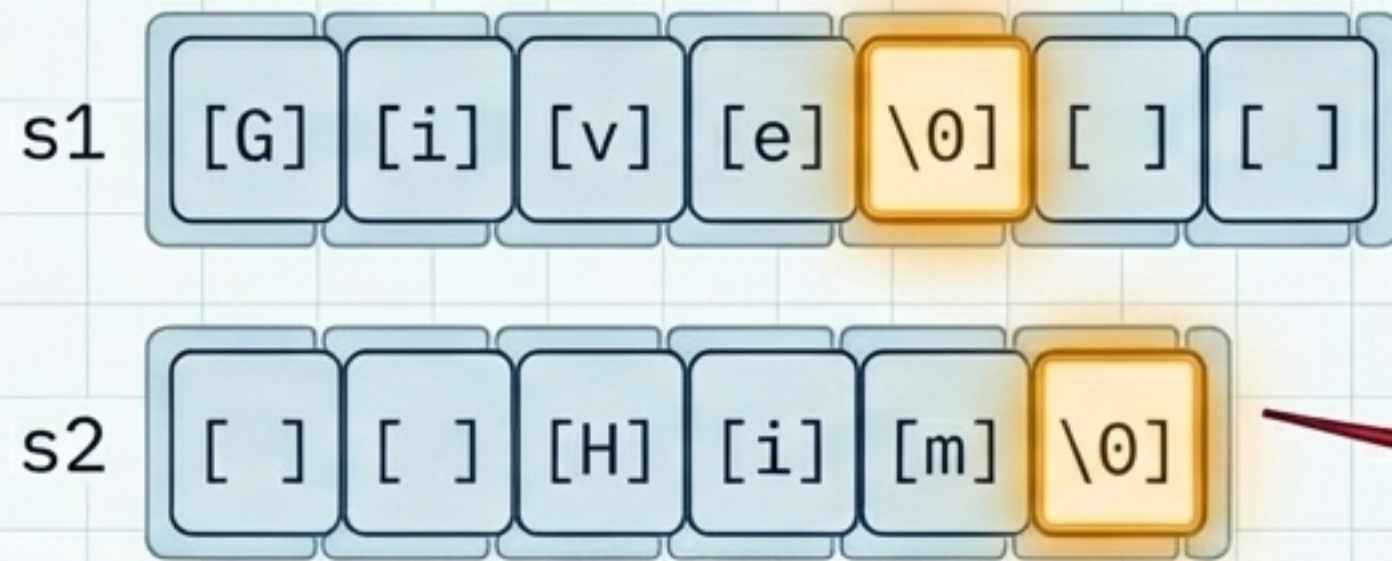


C stringleri karanlıkta ilerler. Sınırlarını bilmezler—ta ki `\0` (Null) karakterine çarpana kadar. Bu karakter silinirse, programınız sonsuz bir uçuruma düşer.

Birleştirme Sanatı ve Taşma Riski: strcat()

```
char *strcat(char *s1, const char *s2);
```

Adım 1 (Önce)



Birleştirme

Adım 2 (Eylem)



strcat(), s2'yi s1'in sonuna eklerken **s1'in kapasitesini kontrol etmez**. Yeterli alan yoksa, size ait olmayan belleği ezmeye başlarsınız.

Stack Bellek Düzenine Güvenilmez

```
char give[16];  
char give[16];  
char him[5];  
char six[5];
```

give: 0x7ffd6ff3ed90

him: 0x7ffd6ff3eda0



Pointer Aritmetiği Tehlikesi: Stack üzerindeki değişkenlerin sırası ve adres yerleşimleri taşınabilir (portable) değildir. İşletim sistemi veya derleyici bu sırayı Hafızada taşıma yapmak, komşu değişkenlerin sessizce bozulmasına yol açar.

100 Kat Daha Yavaş Bir Algoritma: $O(n^2)$ Tuzağı

100,000 Dize İşlemi Süresi

C++ std::string (s += 'j') → 0.01 saniye (Göz açıp kapayıncaya kadar)

C strcat(s, 'j') 3.47 saniye (Neredeyse 100 kat daha yavaş!) →

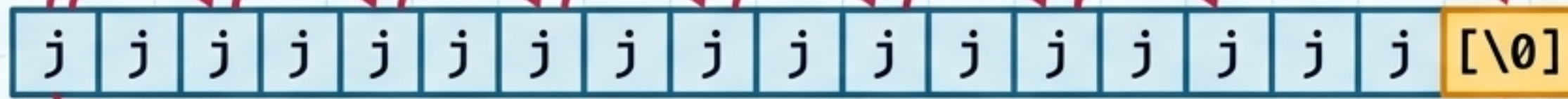
0.0

1.00

2.00

3.45

4.00



Neden Yavaş?

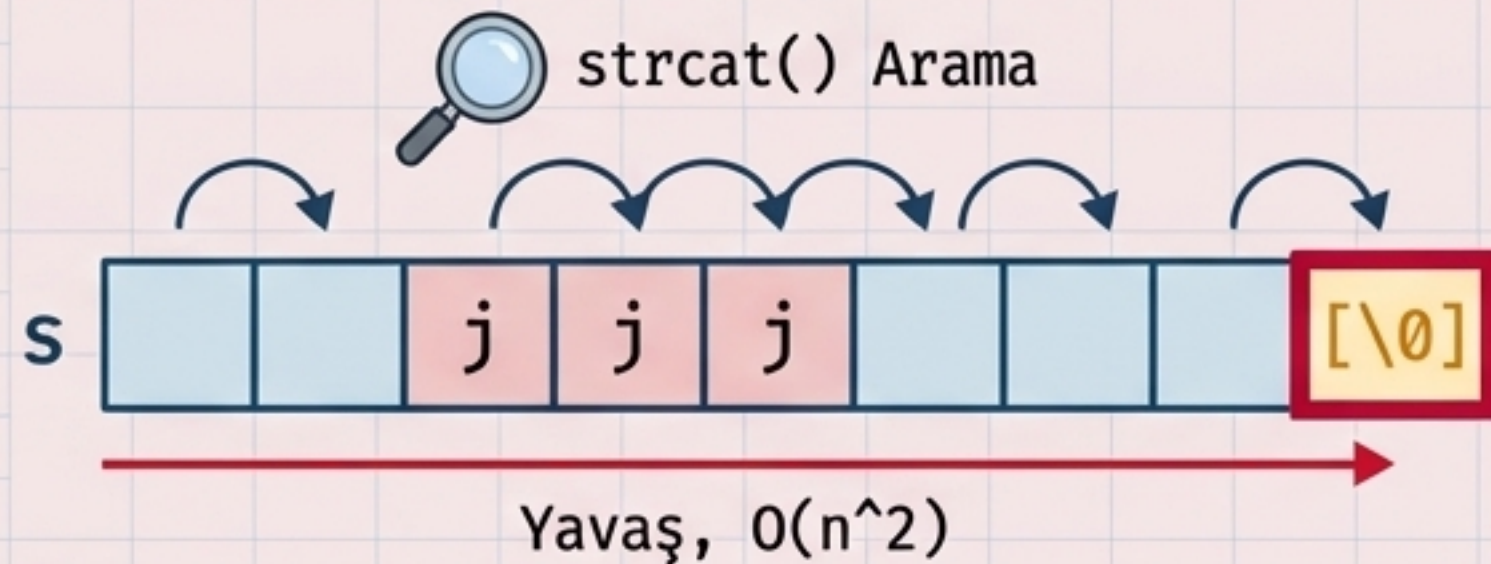
strcat() belleğin neresinde olduğunu hatırlamaz. Her çağrıldığında dizenin sonundaki `\0` karakterini sıfırdan, en baştan başlayarak aramak zorundadır.

O(n) Çözümü: Pointer Aritmetiği

Amatör Yaklaşım



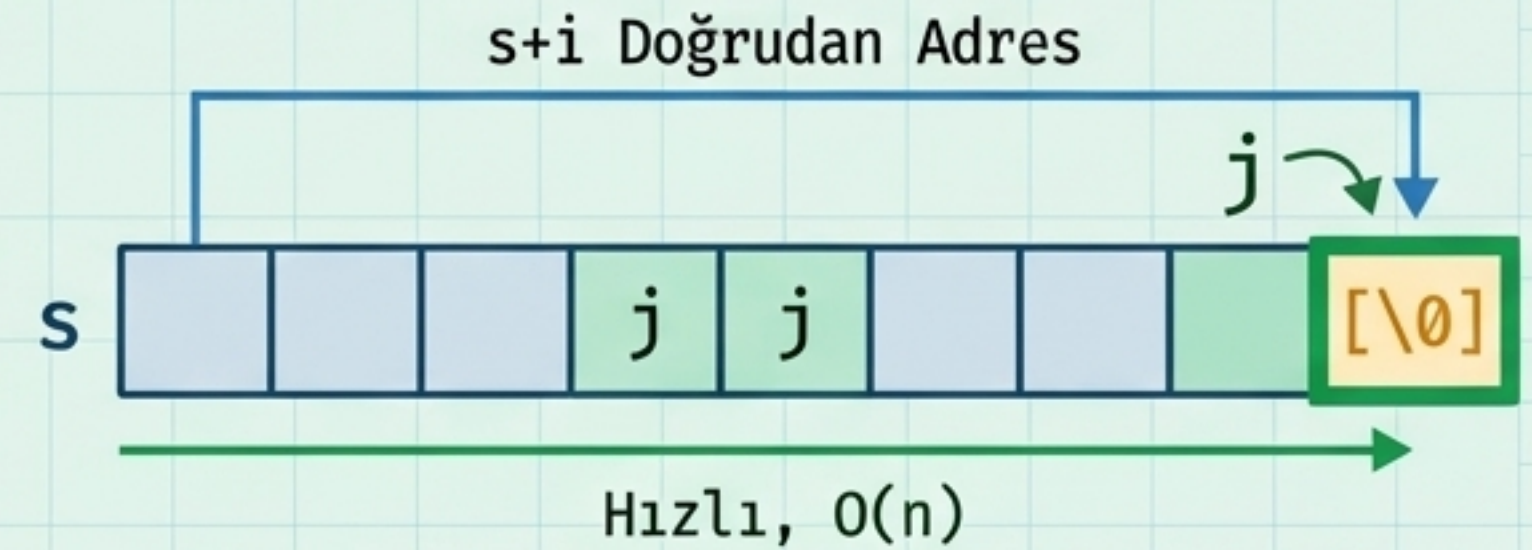
```
for (i = 0; i < n; i++)  
    strcat(s, 'j');
```



Sistem Programcısı Yaklaşımı



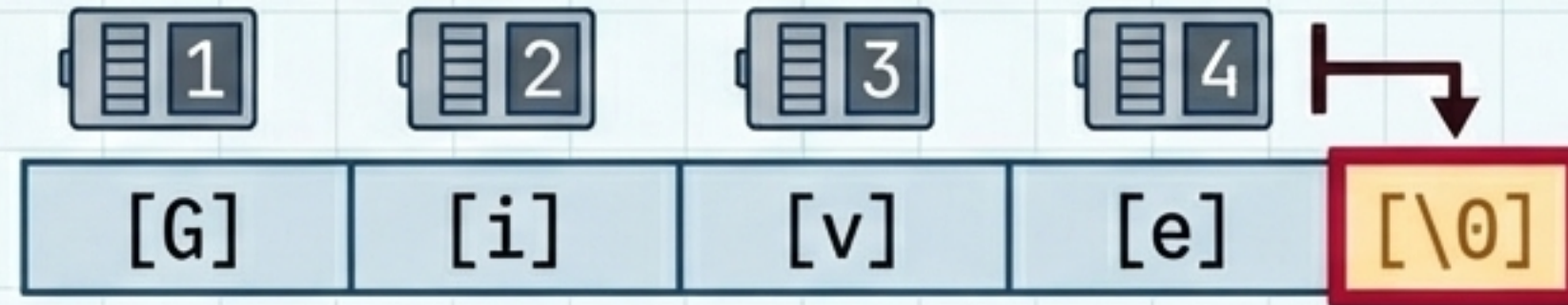
```
for (i = 0; i < n; i++)  
    strcat(s+i, 'j');
```



String'in sonunun nerede olduğunu döngü indeksi (i) sayesinde zaten biliyoruz.
Sisteme adresi doğrudan (s+i) şeklinde vererek arama maliyetini sıfırlayın.

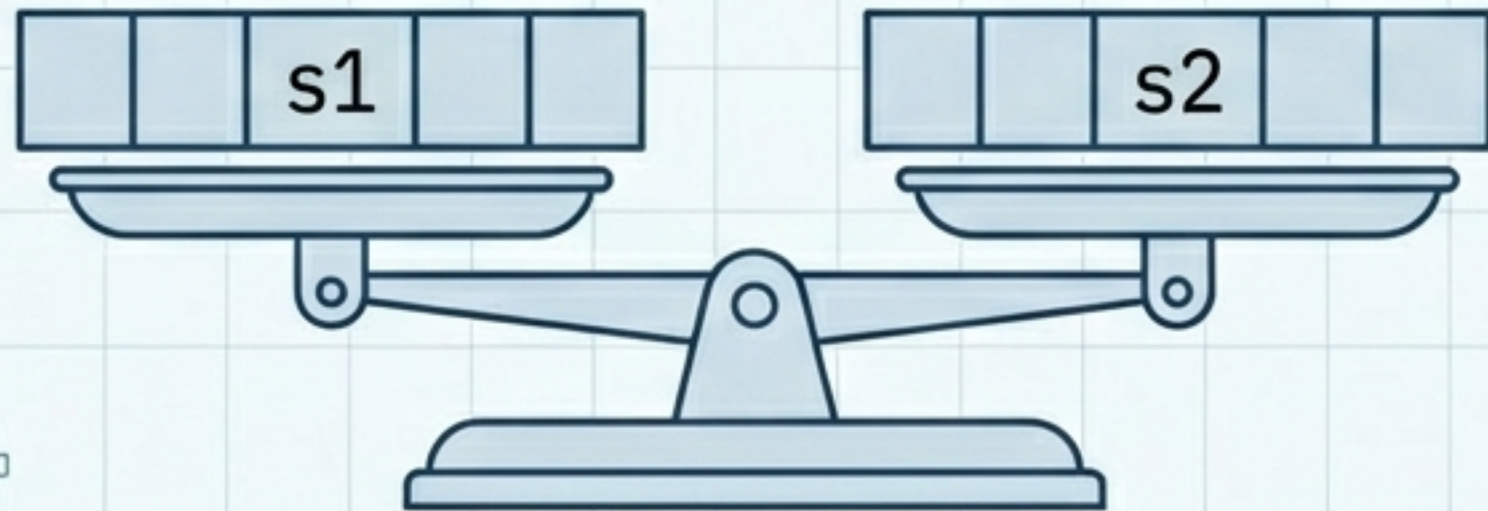
Analiz ve Kıyaslama Ölçütleri

Uzunluk Ölçümü: `strlen()`



Sadece `\0` karakterine kadar olan uzunluğu sayar, null karakterini hesaba katmaz.

Sözlüksel Kıyaslama: `strcmp()`

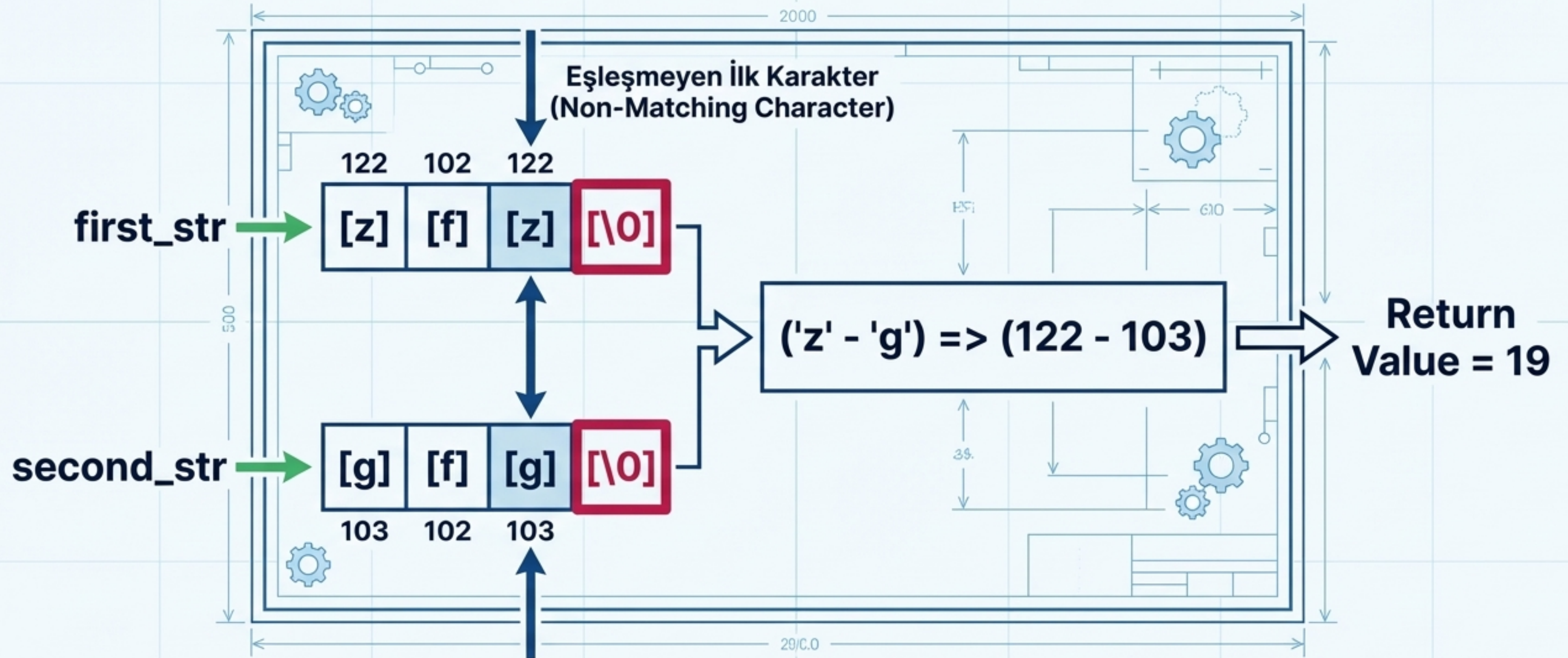


- Eşitlerse: 0 döndürür.
- $s1 < s2$ ise: Negatif sayı döndürür.
- $s1 > s2$ ise: Pozitif sayı döndürür.



Limitli Kıyaslama: `strncmp()` ile sadece ilk 'n' karakterin ölçülmesini sağlayarak güvenliği artırabilirsiniz.

strcmp()'nin Gizli ASCII Motoru



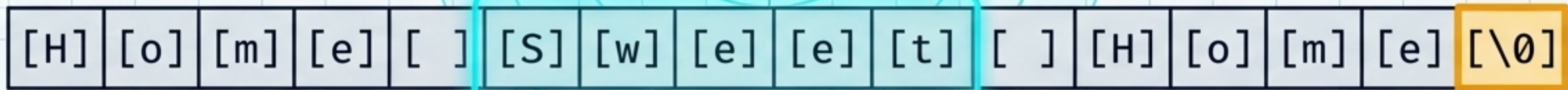
`strcmp()` sadece -1 veya 1 döndürmez. Eşleşmeyen ilk karakteri bulduğunda, bu karakterlerin ASCII değerleri arasındaki matematiksel farkı döndürür.

Bellekte Hedef Bulucu: strchr ve strstr

```
ptr = strchr(line, ' ');
```

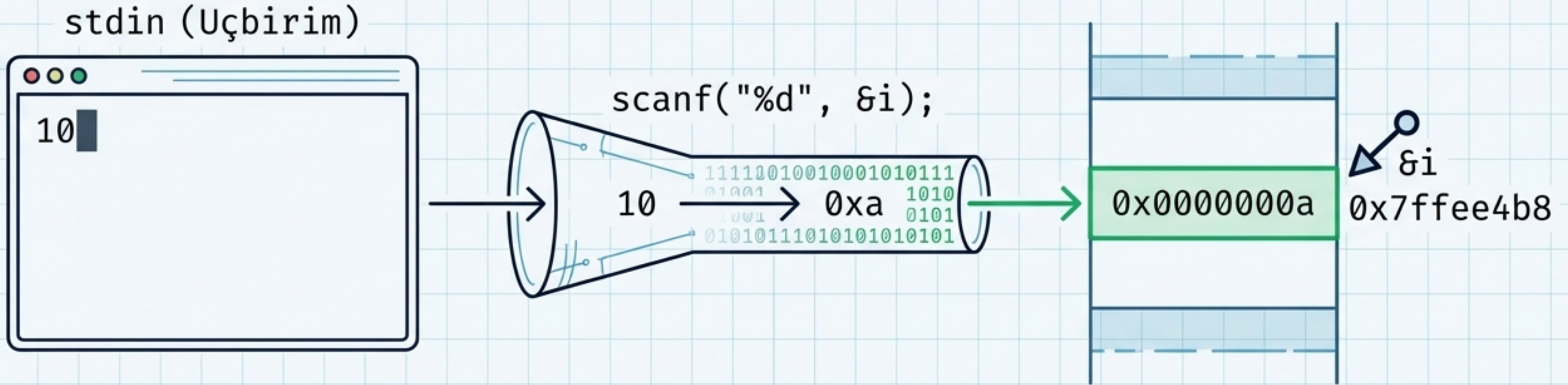


```
strstr(s1, "Sweet");
```



Kritik Detay: Bu fonksiyonlar bir indeks numarası **değil, doğrudan** hedefin bellekteki fiziksel başlangıç adresini (pointer) döndürür. Hedef bulunamazsa NULL döner.

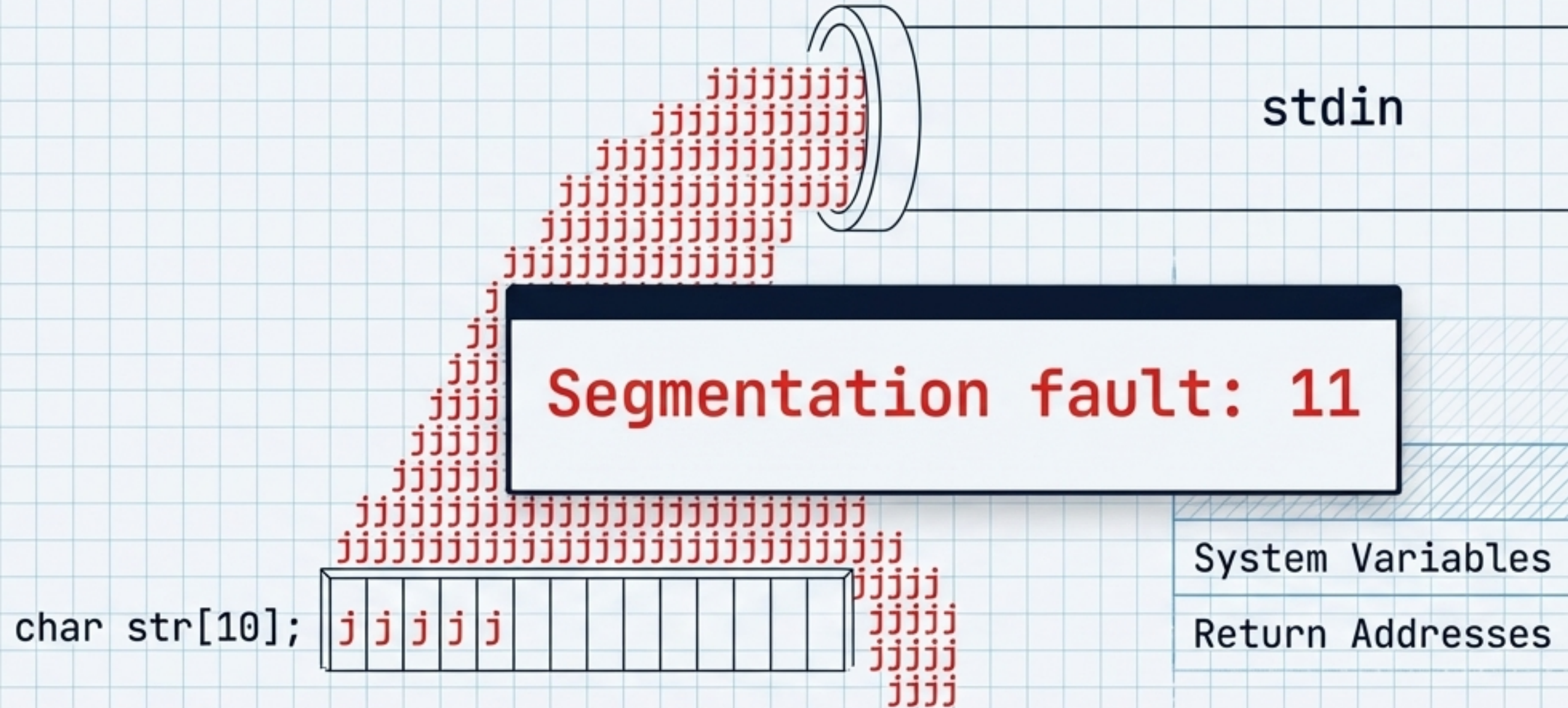
Uçbirimden Belleğe Veri Akışı: scanf()



Neden '&' (Adres Operatörü) Kullanıyoruz?

C dilinde referans değişkenleri yoktur. `scanf()`'in okuduğu değeri yazabilmesi için değişkenin kopyasına değil, bellekteki **fiziksel adresine (&)** **doğrudan erişimi** olması şarttır.

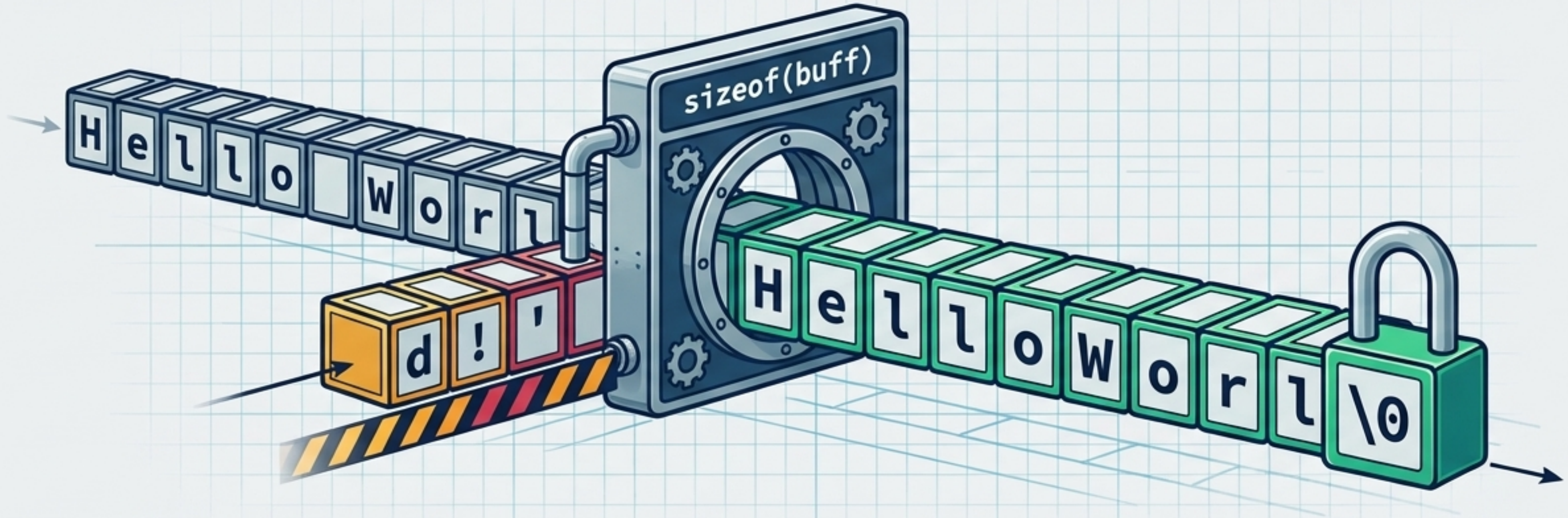
⚠ 80.000 'j' Problemi ve Bellek Taşması



`scanf("%s", str)` sınırları kontrol etmez. Ayrılan 10 byte'lık bellekten fazlası girildiğinde (Buffer Overflow), işletim sistemi güvenliği sağlamak için programı acımasızca sonlandırır.

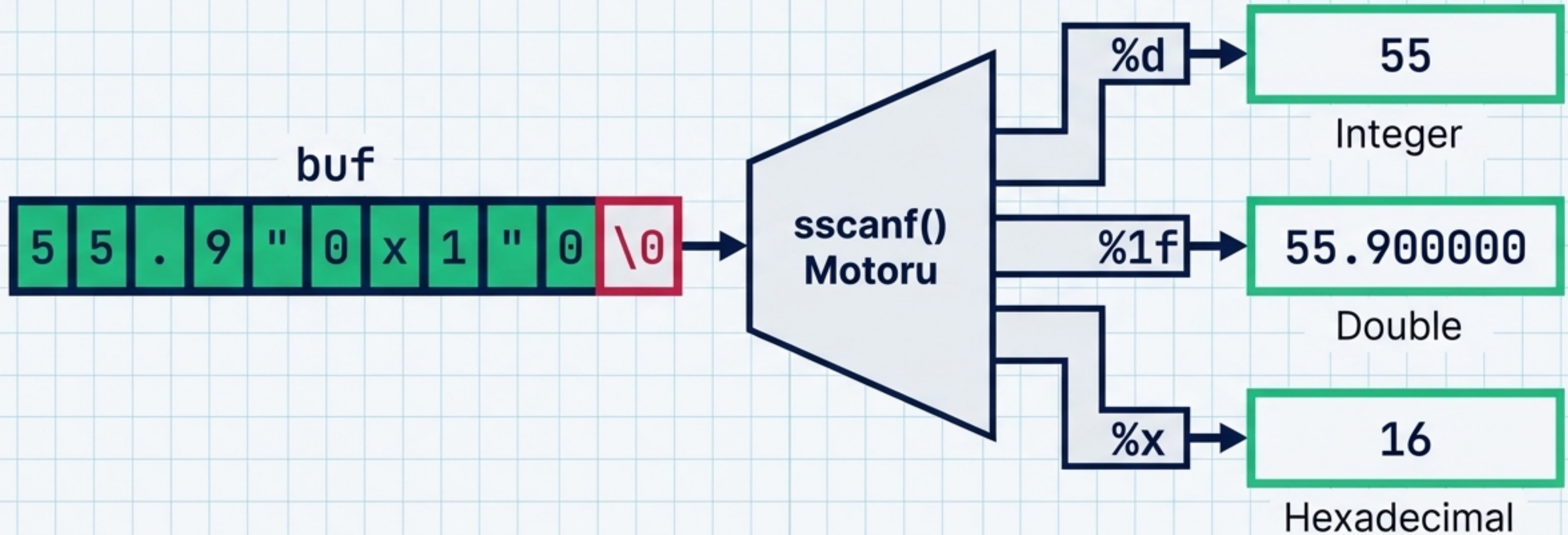
Güvenli Girdi Limanı: fgets()

```
fgets(buff, sizeof(buff), stdin);
```



C'de güvenli metin okumanın altın standardı. Belirtilen boyuta (sizeof), yeni satır karakterine (\n) veya dosya sonuna (EOF) ulaşana kadar okur. Kapıyı daima güvenli bir şekilde \0 ile kilitler ve bellek taşmalarını imkansız hale getirir.

Bellek İçi Veri Çıkartma (Parsing): sscanf()

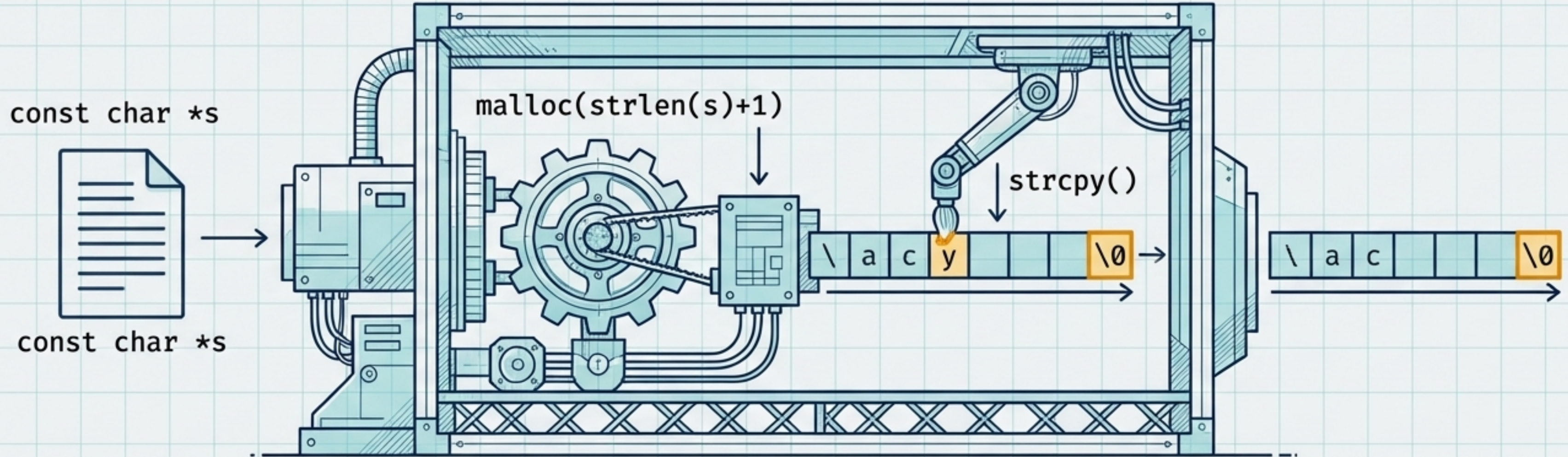


Temel Fark: `scanf()` dışarıdaki terminali dinler. `sscanf()` ise halihazırda bellekte duran bir string'i parçalayıp, içinden tipli verileri (int, double, hex) ayıklamak (parsing) için kullanılır.

I/O Güvenlik ve Karar Matrisi

Fonksiyon	Girdi Kaynağı	Taşıma Riski (Overflow)	Kullanım Senaryosu	Sistem Programcısı Puanı
scanf() (Fira Code)	stdin	YÜKSEK	Sınırlı ve formatlı veri okuma	★★ (Sınırlandırılmamış %s asla kullanılmamalı)
sscanf() (Fira Code)	Bellek (String)	DÜŞÜK	Metin içinden tip dönüşümü (parsing)	★★★★
fgets() (Fira Code)	Dosya / stdin	YOK	Ham ve güvenli metin/satır okuma	★★★★★ (Altın Standart)

Bellek Klonlama ve Sızıntı Riski: strdup()



Gizli Tahsisat (Hidden Allocation): `strdup()` arka planda otomatik olarak `malloc()` kullanır. Klonlanan veriyle işiniz bittiğinde belleği sisteme iade etmek için `free()` çağırmanız zorundasınız. Aksi takdirde **Memory Leak (Bellek Sızıntısı)** oluşur.

Sistem Programcısının Araç Kutusu

Arama (Search)

- **strchr()** : Karakter bulur (Pointer döner)
- **strstr()** : Alt-string (substring) bulur
- **strrchr()** : Sondan başlayarak karakter bulur

Karşılaştırma (Compare)

- **strcmp()** : Sözlük sırasını kıyaslar
- **strncmp()** : Kısıtlı uzunlukta kıyaslar
- **memcmp()** : Ham bayt seviyesinde kıyaslar

Manipülasyon (Manipulate)

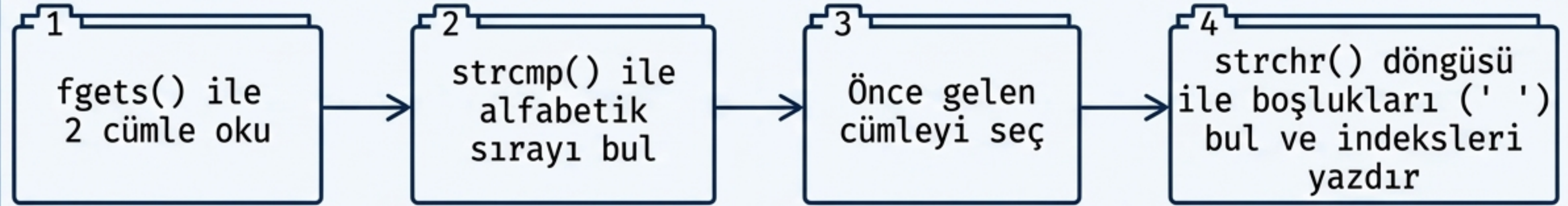
- **strcat()** : Dizenin ucuna ekler
- **strcpy()** : Dizenin üzerine yazar
- **strsep()** : Sınırlayıcıyla kelimelere böler

Bellek Yönetimi (Memory)

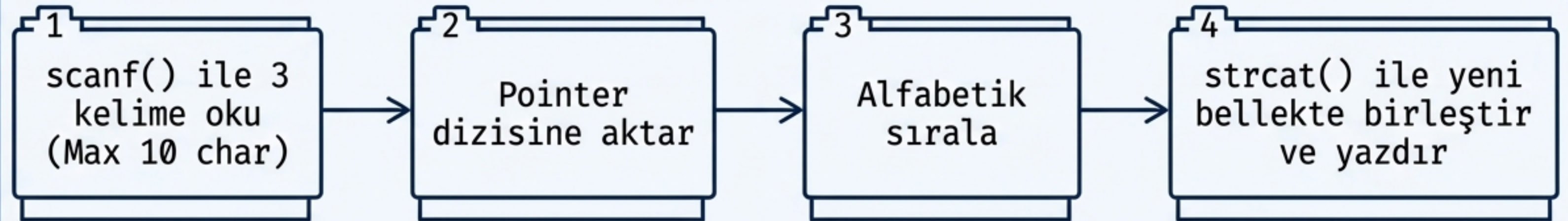
- **memcpy()** : Adres bloğunu kopyalar
- **bzero()** : Bellek bölgesini sıfırlar
- **strdup()** : Yeni bellek ayırıp klonlar

Bellek Mimarisi Meydan Okuması

Mission 1



Mission 2



```
root@sys:~/memory_layout# make && ./challenge
```